Tomato Leaf Disease Detection

Statistical Methods in AI — Assignment 3

Problem: Image Classification · Transfer Learning · Web Application

Arun K
2025121004
arun.k@research.iiit.ac.in

Daniel Ashish Abraham
2025121010
daniel.abraham@research.iiit.ac.in

Source Code: `github.com/ashdane/nineLives-SMAI-A3`

1. Introduction

Crop disease accounts for a substantial fraction of annual yield loss in Indian agriculture. Tomatoes, one of the most widely cultivated vegetables, are susceptible to over a dozen fungal, bacterial, and viral diseases, several of which can destroy an entire field within days if left untreated. The traditional response wait for an agronomist, send a photo to a call centre, or consult a printed guide is too slow and too inaccessible for smallholder farmers operating under time and resource pressure.

This project addresses that gap with an end-to-end automated system. A farmer photographs a leaf using their phone; the application names the disease within seconds and tells them exactly what to do next, in a language they can read. The core technical challenge is two-fold: first, the classifier must be robust to the messy, variable-quality images that come from smartphone cameras in field conditions; second, the system must handle invalid inputs gracefully, since any image-to-class model will hallucinate a plausible-sounding disease label even when the input is a photograph of a car.

The solution combines two pre-trained vision models in a sequential pipeline. OpenAI CLIP performs zero-shot validation to gate invalid inputs before they reach the classifier. A fine-tuned EfficientNet-B0 then performs the actual disease classification over ten tomato-specific classes. The result is deployed as a Streamlit web application with multi-language support covering English, Hindi, Tamil, Telugu, and Malayalam.

The remainder of this report is structured as follows. Section 2 describes the dataset and augmentation strategy. Section 3 covers both model architectures and the training procedure in detail. Section 4 describes the system architecture and pipeline design. Section 6 presents results and application screenshots. Section 7 contains an ablation study. Section 11 discusses limitations and Section 12 lists references.

2. Data

2.1. Dataset

The PlantVillage dataset [1], filtered to the ten tomato disease classes, serves as the training corpus. Table 1 summarises class distribution. The dataset presents two challenges worth noting: all images are captured under controlled laboratory conditions with uniform backgrounds, and the class distribution is imbalanced by a factor of roughly 14× between the largest class (Yellow Leaf Curl Virus, ∼5,357 images) and the smallest (Mosaic Virus, ∼373 images).

Table 1: Dataset class distribution (PlantVillage – Tomato subset)

Class	**Severity**	**Approx. Images**
Tomato – Healthy	None	∼1,590
Tomato – Bacterial Spot	High	∼2,127
Tomato – Early Blight	High	∼1,000
Tomato – Late Blight	Critical	∼1,909
Tomato – Leaf Mold	Medium	∼952
Tomato – Septoria Leaf Spot	High	∼1,771
Tomato – Spider Mites	Medium	∼1,676
Tomato – Target Spot	Medium	∼1,404
Tomato – Yellow Leaf Curl Virus	High	∼5,357
Tomato – Mosaic Virus	High	∼373
Total		**∼18,159**

The dataset is split 80/20 into training and validation sets with stratified sampling to preserve class ratios.

2.2. Data Augmentation

PlantVillage images are controlled-environment photographs with consistent lighting, framing, and backgrounds. Field-captured images are nothing like that — they arrive at different angles, distances, lighting conditions, and with partial occlusion from other leaves. To narrow the distribution gap, the training pipeline applies the following augmentations stochastically during each epoch:

- `RandomResizedCrop(224)` with scale $(0.75, 1.0)$
- `RandomHorizontalFlip` ($p = 0.5$) and `RandomVerticalFlip` ($p = 0.3$)
- `RandomRotation` ($\pm 20°$)
- `ColorJitter` (brightness ± 0.25, contrast ± 0.25, saturation ± 0.2, hue ± 0.05)
- `Normalize` with ImageNet mean $\mu = [0.485, 0.456, 0.406]$ and std $\sigma = [0.229, 0.224, 0.225]$

Validation uses only `Resize(256)` → `CenterCrop(224)` → `Normalize`, ensuring that

the reported accuracy reflects clean inference and does not benefit from test-time augmentation.

3. Method

The system uses two models in sequence: CLIP for input validation and EfficientNet-B0 for disease classification. This section describes both architectures in depth, along with training details for the classifier.

3.1. Model 1: CLIP for Zero-Shot Input Validation

3.1.1. Architecture

CLIP (Contrastive Language–Image Pre-training) [2] is a dual-encoder model developed by OpenAI and trained on approximately 400 million image–text pairs collected from the internet. It consists of two separate encoders:

- **Image encoder:** A Vision Transformer ViT-B/32, which divides the input image into 32×32 non-overlapping patches, linearly projects each patch into a 768-dimensional token, prepends a learnable `[CLS]` token, and passes the sequence through 12 transformer blocks. The output is a 512-dimensional image embedding obtained by projecting the `[CLS]` representation.
- **Text encoder:** A GPT-2-style transformer with 12 layers and a context length of 77 tokens. The text prompt is tokenised using a byte-pair encoding vocabulary, and the `[EOS]` token representation is projected to a 512-dimensional text embedding.

Both encoders project into a shared 512-dimensional embedding space. CLIP is trained with a symmetric contrastive loss that pushes matching image–text pairs together and pulls non-matching pairs apart:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N}\sum_{i=1}^{N}\left[\log\frac{\exp(\cos(v_i, t_i)/\tau)}{\sum_{j=1}^{N}\exp(\cos(v_i, t_j)/\tau)} + \log\frac{\exp(\cos(v_i, t_i)/\tau)}{\sum_{j=1}^{N}\exp(\cos(v_j, t_i)/\tau)}\right] \tag{1}$$

where v_i and t_i are the normalised image and text embeddings for sample i, and τ is a learnable temperature parameter.

3.1.2. Zero-Shot Classification Mechanism

CLIP's zero-shot capability comes from the alignment of its embedding space across modalities. To classify an image against a set of candidate text descriptions, cosine similarities between the image embedding and each text embedding are computed, then converted to probabilities via softmax:

$$P(\text{class } k \mid \text{image}) = \frac{\exp(\cos(v, t_k)/\tau)}{\sum_j \exp(\cos(v, t_j)/\tau)} \tag{2}$$

In this system, three candidate prompts are used:

1. "a photo of a tomato plant leaf, with or without disease"
2. "a photo of a non-tomato plant leaf such as banana, papaya, mango"
3. "a photo of something that is not a plant leaf"

The softmax probability assigned to prompt 1 is used as the "tomato leaf confidence" score. No fine-tuning of CLIP is performed; the model is used entirely off-the-shelf. The model variant used is `openai/clip-vit-base-patch32`, loaded via the HuggingFace `transformers` library.

3.1.3. Gate Thresholds

Based on empirical testing on a held-out set of non-leaf images, a three-tier threshold scheme is applied:

Table 2: CLIP gate decision thresholds

Tomato Leaf Prob.	**Decision**	**User Experience**
$< 20\%$	Reject	Error: "This does not appear to be a tomato leaf"
20%–50%	Warn	Warning with "Continue anyway" and "Re-upload" buttons
$> 50\%$	Accept	Proceed directly to disease classification

3.2. Model 2: EfficientNet-B0 for Disease Classification

3.2.1. Architecture

EfficientNet [3] is a family of convolutional neural networks designed around a compound scaling principle. Rather than scaling depth, width, or resolution independently (as ResNet and VGG variants do), EfficientNet scales all three dimensions jointly according to a fixed ratio derived from a neural architecture search. The scaling coefficients (α, β, γ) satisfy:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \quad \text{subject to } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \tag{3}$$

where d, w, r denote depth, width, and resolution scaling factors, and ϕ is a compound coefficient. EfficientNet-B0 is the baseline model ($\phi = 0$) with the following profile:

Table 3: EfficientNet-B0 architecture summary

Stage	**Operator**	**Resolution**	**Channels**	**Layers**
Stem	Conv 3×3	224×224	32	1
Stage 1	MBConv1, 3×3	112×112	16	1
Stage 2	MBConv6, 3×3	56×56	24	2
Stage 3	MBConv6, 5×5	28×28	40	2
Stage 4	MBConv6, 3×3	14×14	80	3
Stage 5	MBConv6, 5×5	14×14	112	3
Stage 6	MBConv6, 5×5	7×7	192	4
Stage 7	MBConv6, 3×3	7×7	320	1
Head	Conv 1×1, Pool	7×7	1280	1
Classifier	FC (10 classes)	—	10	1

The core building block is the Mobile Inverted Bottleneck Convolution (MBConv), which expands the channel dimension by a factor of 6, applies a depthwise separable convolution, and then projects back down. Squeeze-and-Excitation (SE) modules are embedded within each block to recalibrate channel-wise feature responses. The model has approximately 5.3 million parameters.

3.2.2. Transfer Learning and Two-Phase Fine-Tuning

The model is initialised from ImageNet-pretrained weights, loaded via the `timm` library. Fine-tuning follows a two-phase strategy:

Phase 1 — Head Warmup (Epoch 1). The backbone (all stages except the final classifier layer) is frozen. Only the classification head is trained using Adam with learning rate 10^{-3} and weight decay 10^{-4}. This allows the randomly-initialised head to converge toward the tomato disease distribution before any backbone gradients are computed, preventing large early-epoch gradients from corrupting the pretrained representations.

Phase 2 — Full Fine-Tuning (Epochs 2–3). All layers are unfrozen. The learning rate is dropped to 10^{-4} and a cosine annealing schedule is applied, gradually reducing the rate to near zero. Fine-tuning the backbone at a low rate allows the pretrained features to adapt to disease-specific textures (lesion patterns, necrotic spots, discolouration) while retaining low-level edge and colour detectors that are useful regardless of domain.

Loss function. Cross-entropy with label smoothing $\epsilon = 0.1$ is used. Label smoothing

replaces the hard one-hot target vector with a soft distribution:

$$\tilde{y}_k = \begin{cases} 1 - \epsilon + \epsilon/K & \text{if } k = \text{true class} \\ \epsilon/K & \text{otherwise} \end{cases} \tag{4}$$

where $K = 10$ is the number of classes. This prevents the model from becoming overconfident, which is important here because confidence scores are surfaced directly to the user and ambiguous predictions trigger different UI states.

The model achieves approximately 96% top-1 validation accuracy after three epochs.

4. System Architecture

The application follows a sequential two-stage pipeline in which the CLIP gate precedes the disease classifier. Figure 1 shows the complete architecture and Figure 2 shows the corresponding user flow.

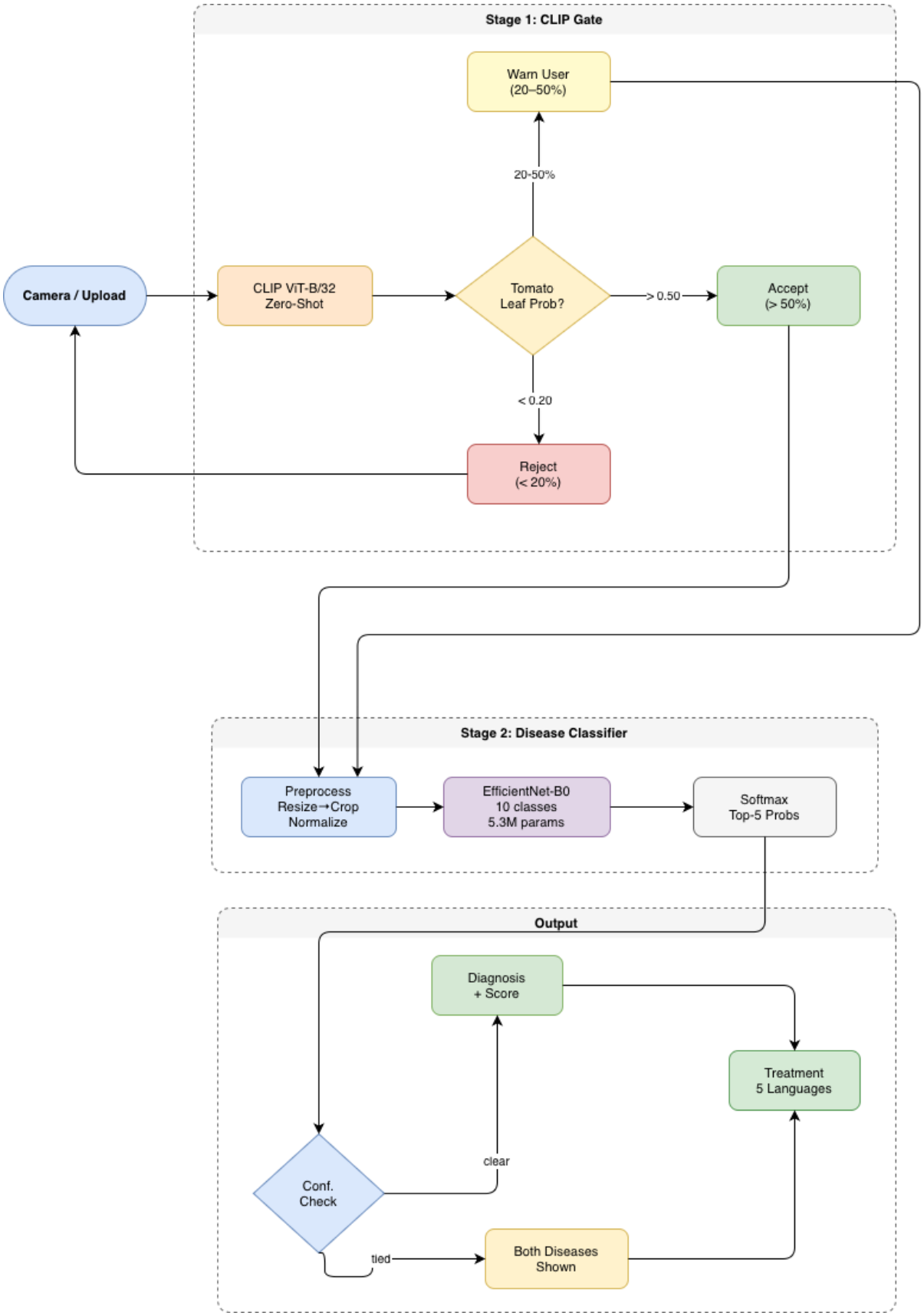


Figure 1: System architecture: two-stage pipeline with CLIP zero-shot gate and EfficientNet-B0 disease classifier. Rejected images loop back to the upload screen.

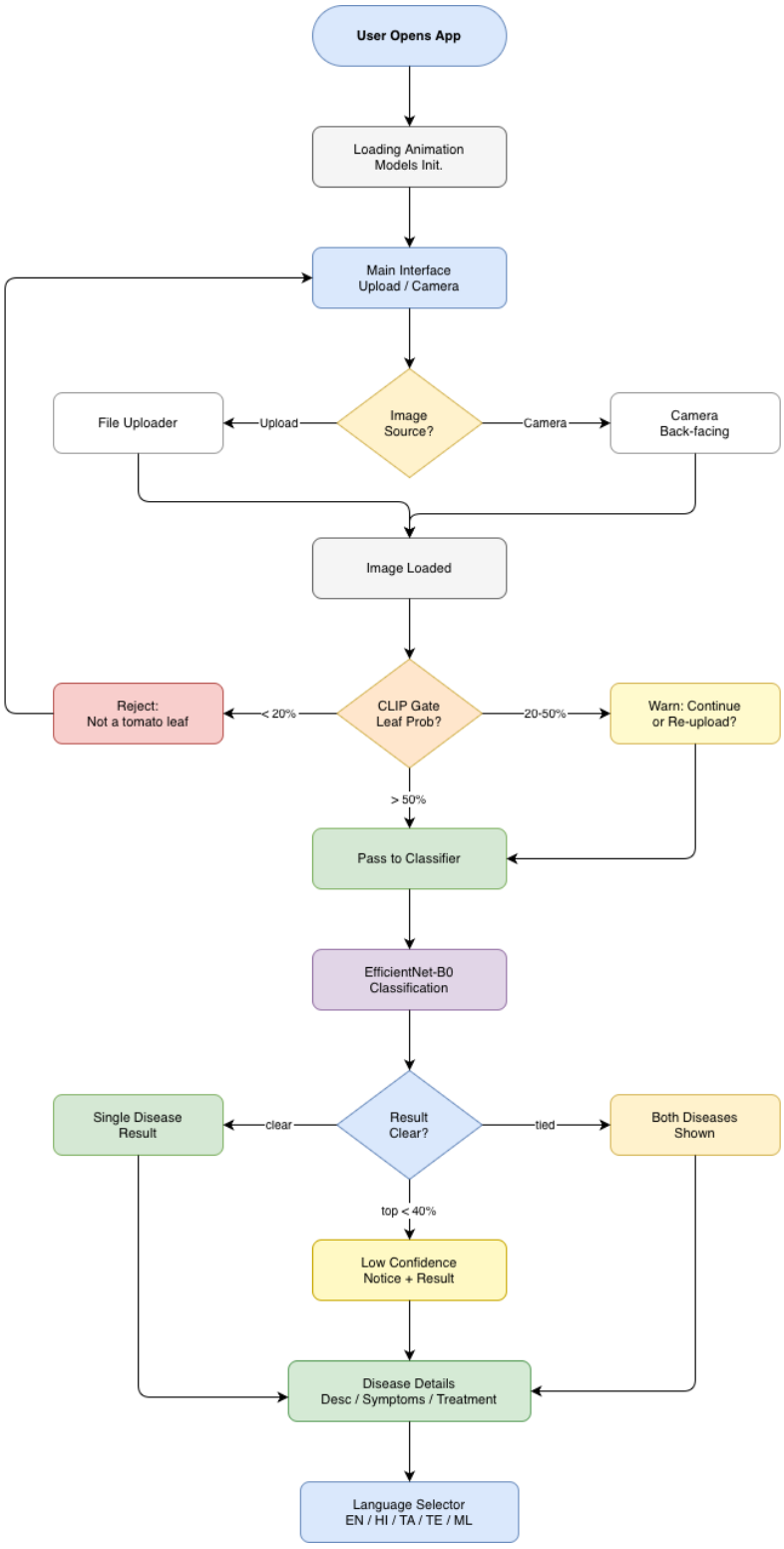


Figure 2: Complete user flow from image input through CLIP validation and EfficientNet-B0 classification to the final disease diagnosis output.

4.1. Stage 1: CLIP-Based Input Validation

Before invoking the disease classifier, the system runs the uploaded image through OpenAI CLIP (`clip-vit-base-patch32`) in zero-shot mode. CLIP computes cosine similarity

between the image embedding and three text prompts:

1. "a photo of a tomato plant leaf, with or without disease"
2. "a photo of a non-tomato plant leaf such as banana, papaya, mango"
3. "a photo of something that is not a plant leaf"

The softmax over these three similarity scores gives a "tomato leaf probability". Based on this score, one of three outcomes is triggered, as described in Table 2.

4.2. Stage 2: Disease Classification

Images that pass the CLIP gate are preprocessed with `Resize(256)` → `CenterCrop(224)` → `Normalize` and passed through the fine-tuned EfficientNet-B0. The output is a softmax probability distribution over the ten classes; the top-5 predictions are displayed with confidence bars.

4.3. Ambiguity Handling

Two failure modes are explicitly handled. When the top prediction falls below 40% confidence, a notice is shown recommending that the user photograph the leaf from a different angle or in better light. When the top two predictions are within 10 percentage points of each other and neither exceeds 60%, both diseases are shown in full detail alongside a request for another photo, so the farmer has actionable information regardless.

5. Application Features

5.1. Multi-Language Support

All disease content name, description, symptoms, causes, and treatment is available in five languages: English (default), Hindi, Tamil, Telugu, and Malayalam. Translations are pre-generated and stored in `disease_info.json`, so switching languages is instant and works entirely offline. This matters because the target users are rural farmers who may have poor or no mobile data connectivity.

5.2. Camera Integration

The app includes a camera button that activates the device's rear camera via a JavaScript override of `getUserMedia` with `facingMode: "environment"`. This allows farmers to photograph a leaf directly from the app without needing to switch to a camera app and re-import the image.

5.3. Disease Information Database

Each of the ten disease classes includes a farmer-readable description, the pathogen or environmental cause, a bulleted list of visual symptoms, structured action items (imme-

diate action, treatment, prevention), and a severity classification (none / medium / high / critical).

6. Results and Demonstration

6.1. Loading Screen

The app initialises both CLIP and EfficientNet-B0 on startup. While the models load, an animated loading screen is shown to prevent the user from interacting with an uninitialised interface.

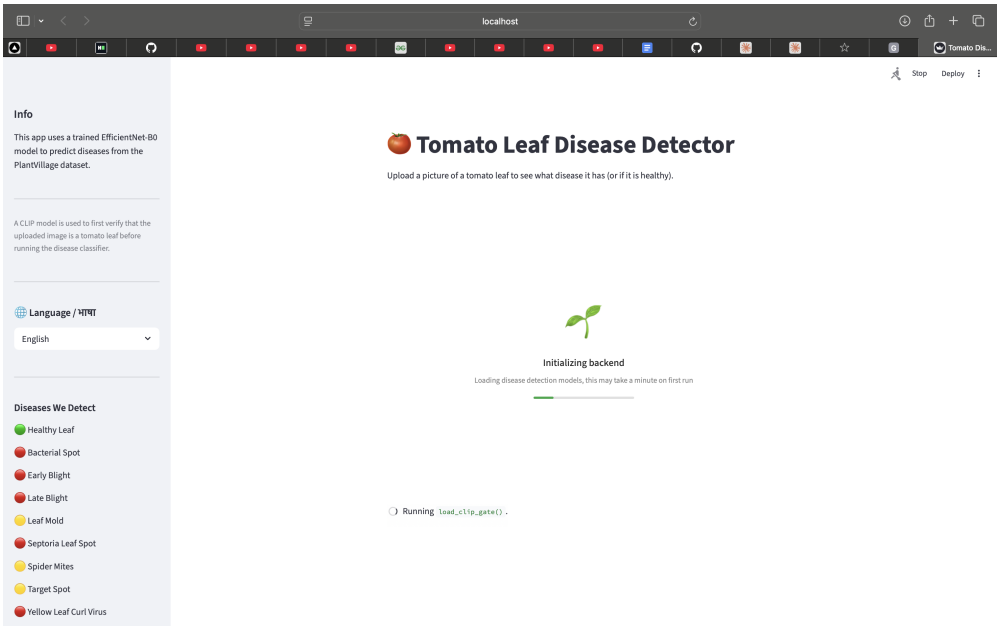


Figure 3: Loading animation displayed while the CLIP and EfficientNet-B0 models initialise in the background

6.2. Main Interface

The main interface shows the upload widget, a camera button, and a sidebar containing an information panel, a language selector, and the list of detectable diseases. This layout keeps all controls accessible without requiring scrolling on most screens.

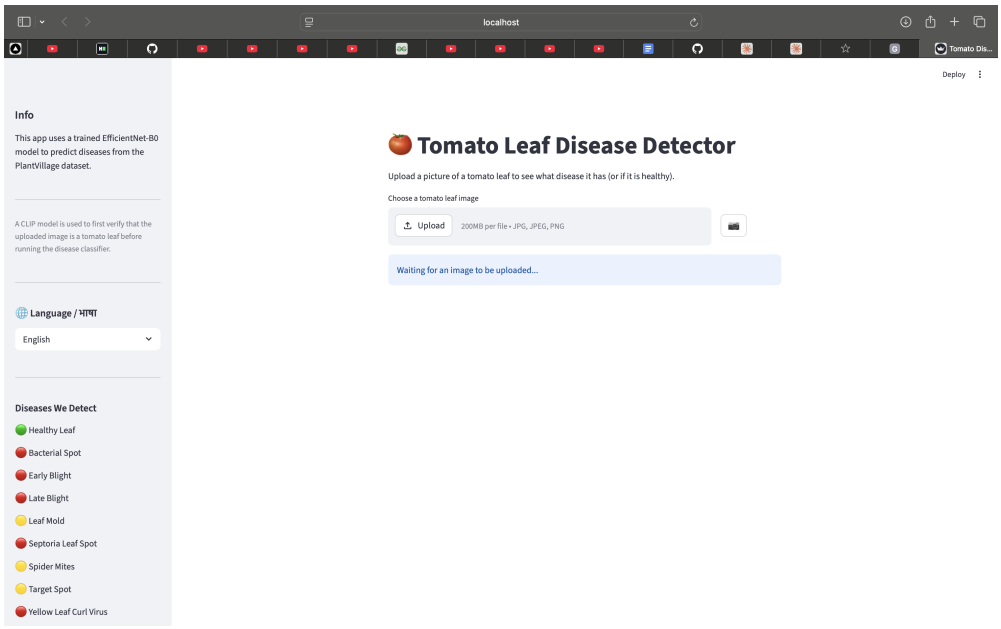


Figure 4: Main interface with file uploader, camera trigger, language selector, and disease reference list in the sidebar

6.3. Healthy Leaf Detection

When a healthy tomato leaf is uploaded, the classifier correctly assigns the highest probability to the "Healthy Leaf" class. The prediction result panel shows the confidence score, and the disease details section below confirms the plant is healthy with no action required.

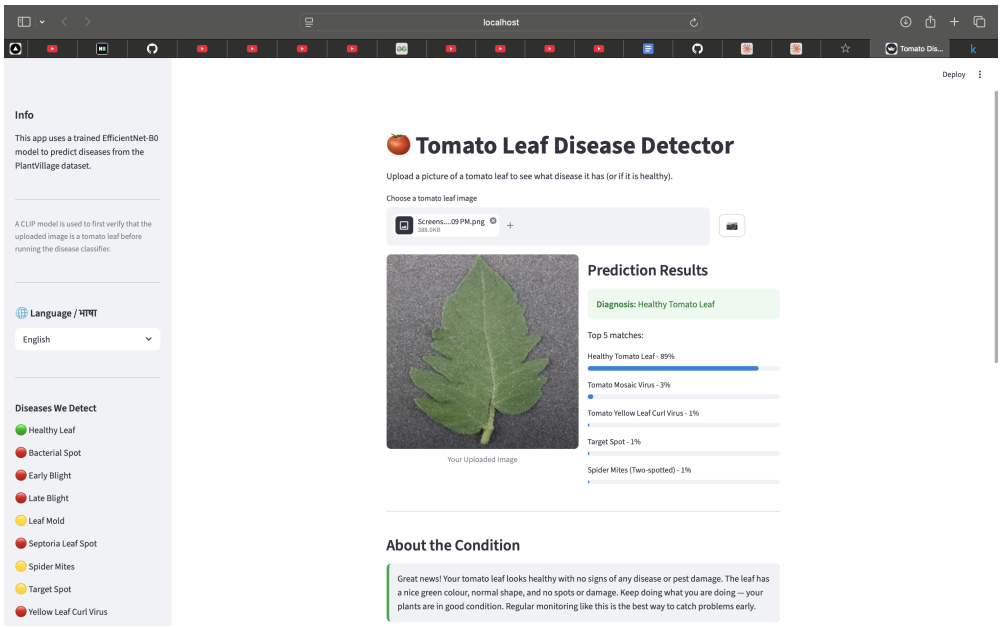


Figure 5: Healthy tomato leaf correctly identified with high confidence

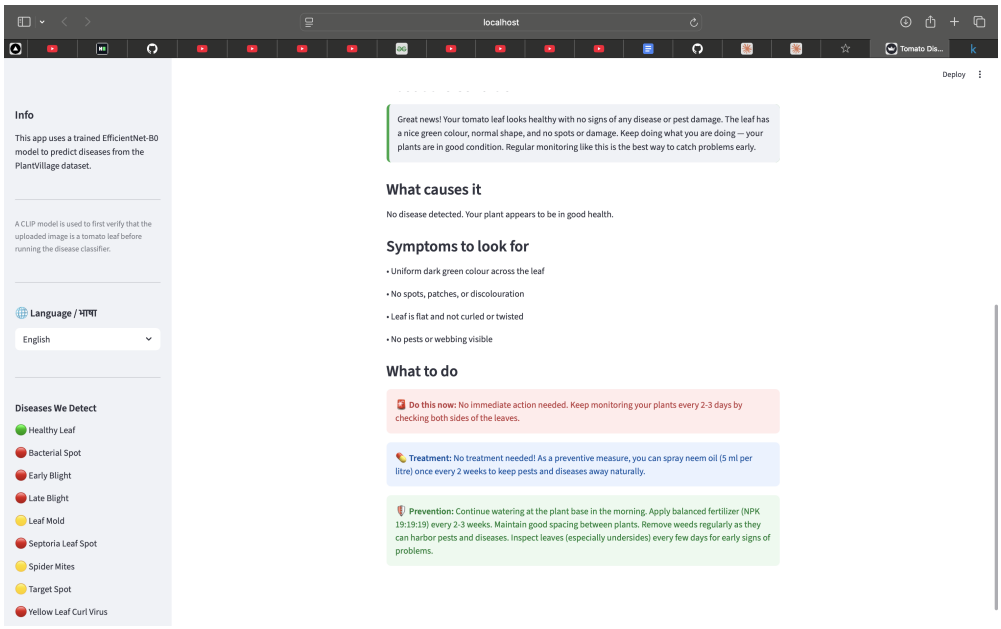


Figure 6: Disease details panel for a healthy leaf, showing description and preventive recommendations

6.4. Disease Detection

Figures 7 through 10 show detection of two diseases: Septoria Leaf Spot and Late Blight. For Septoria, the model outputs 54% probability for the top class, flagged as "Low Confidence" because it falls below 60%. The top-5 breakdown is displayed, reflecting genuine visual similarity between fungal diseases. For Late Blight at 79% confidence, the critical severity level triggers an emergency-framed action panel.

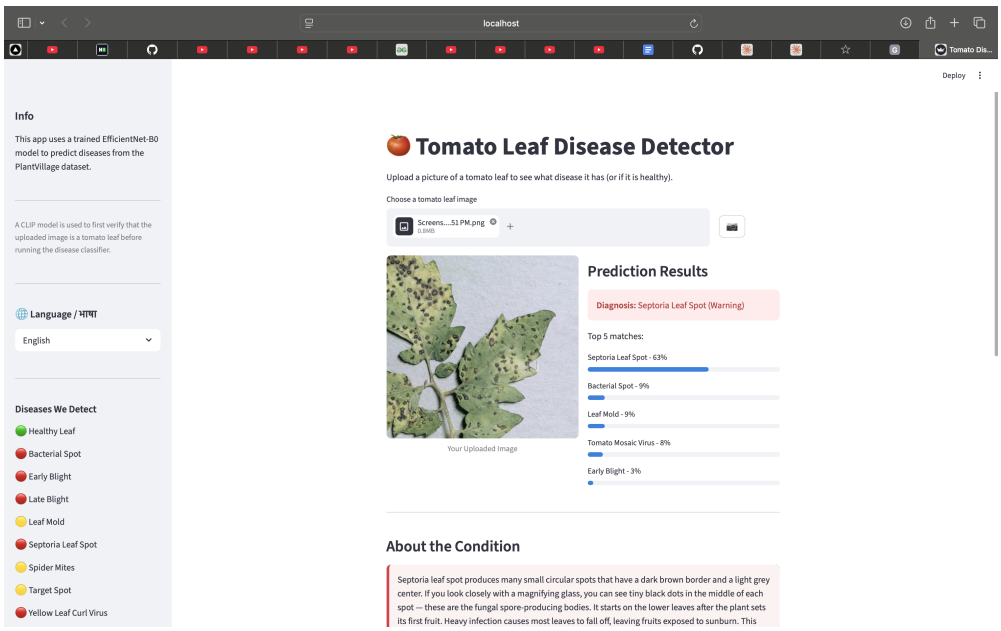


Figure 7: Septoria Leaf Spot detected at 54% confidence; top-5 prediction breakdown shown with confidence bars

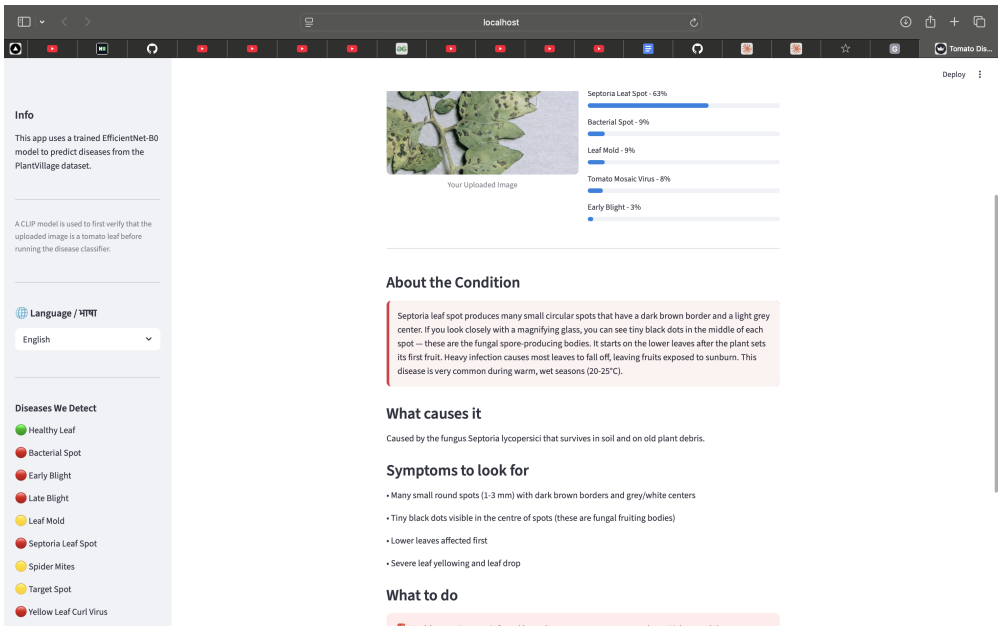


Figure 8: Disease details for Septoria Leaf Spot: description, cause, and visual symptom list

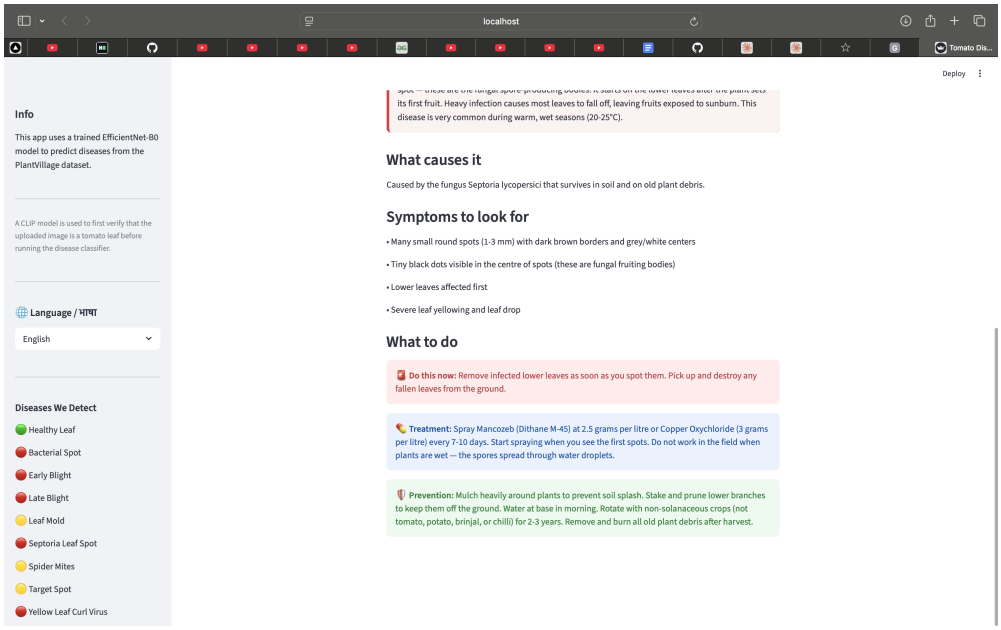


Figure 9: Treatment recommendations for Septoria Leaf Spot, colour-coded by action type

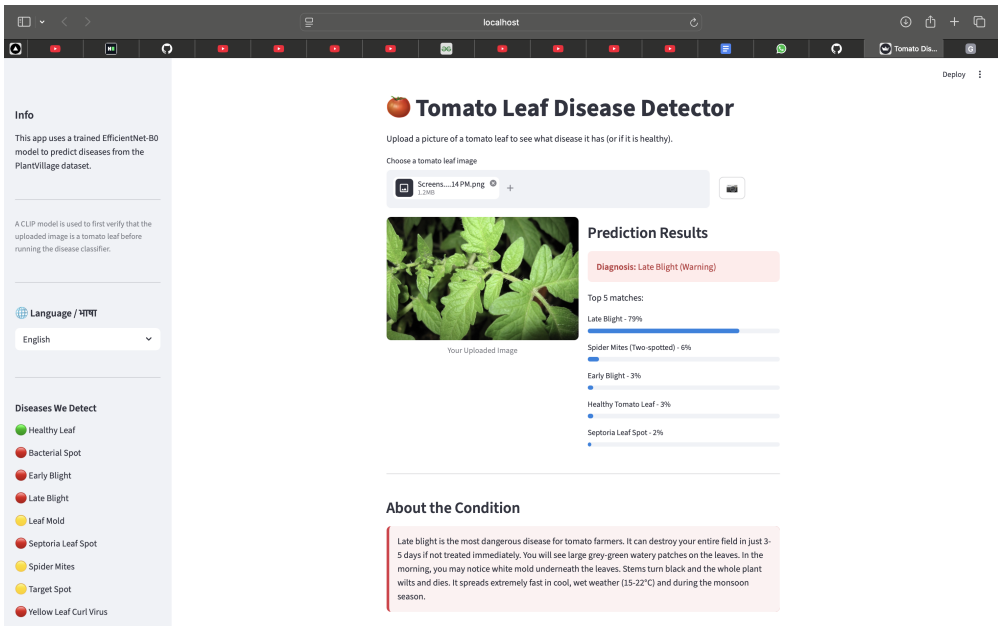
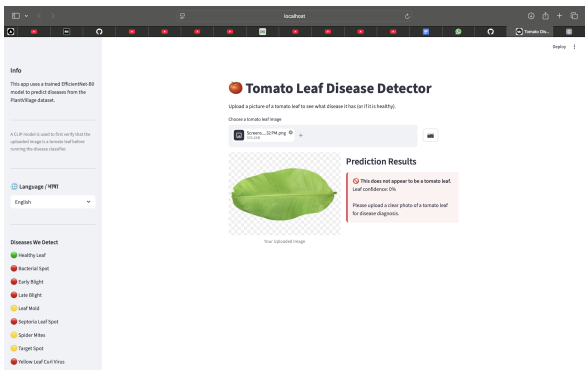


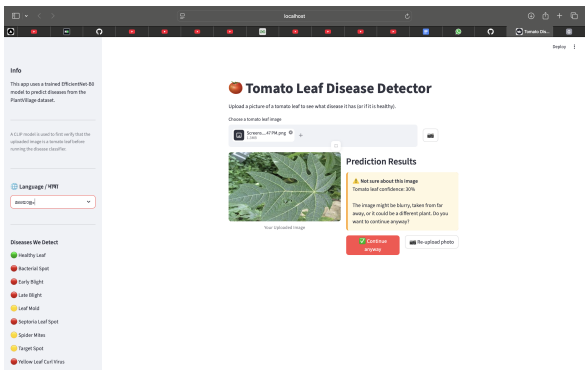
Figure 10: Late Blight detected at 79% confidence; critical severity triggers an emergency action panel

6.5. Non-Tomato Leaf Rejection (CLIP Gate)

A banana leaf is correctly rejected at 0% tomato leaf confidence. A papaya leaf scores 30%, placing it in the uncertain zone; the user is presented with a warning and two options. A camera capture of a laptop surface is rejected at 4% confidence, confirming the gate works on camera-captured images as well as uploads.



(a) Banana leaf rejected (0% confidence)



(b) Papaya leaf in uncertain zone (30%)

Figure 11: CLIP gate in action: hard rejection (left) and three-tier uncertain warning (right)

6.6. Ambiguous Prediction

When the top two predictions are within 10 percentage points, both diagnoses are surfaced. Figures 12–15 show Tomato Yellow Leaf Curl Virus and Late Blight both at 27–28%, triggering the "Multiple possible matches" notice with full information cards for each disease.

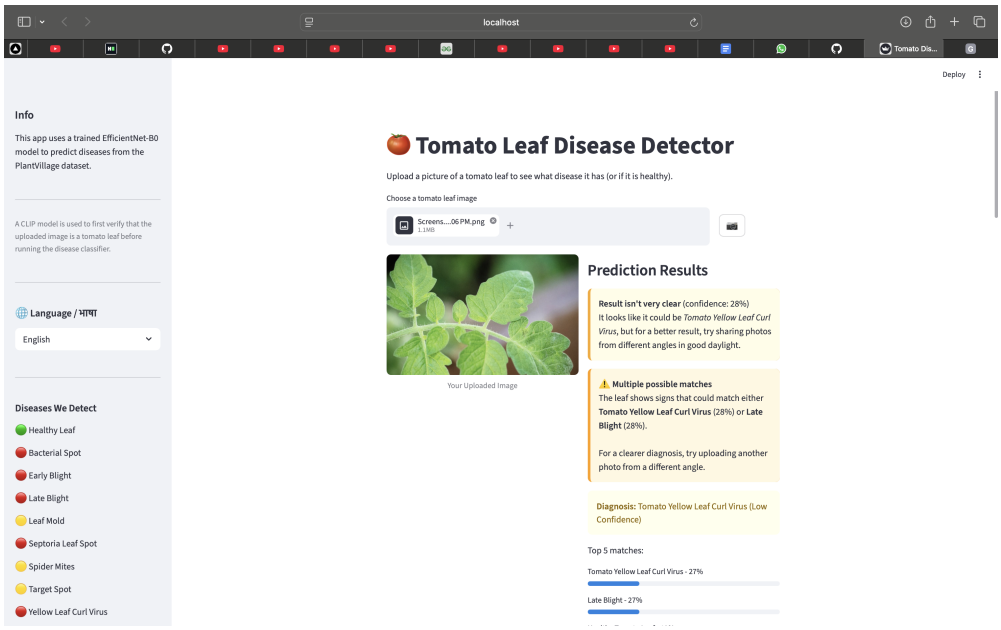


Figure 12: Tied prediction (TYLCV at 27% and Late Blight at 27%) triggers the ambiguous result panel

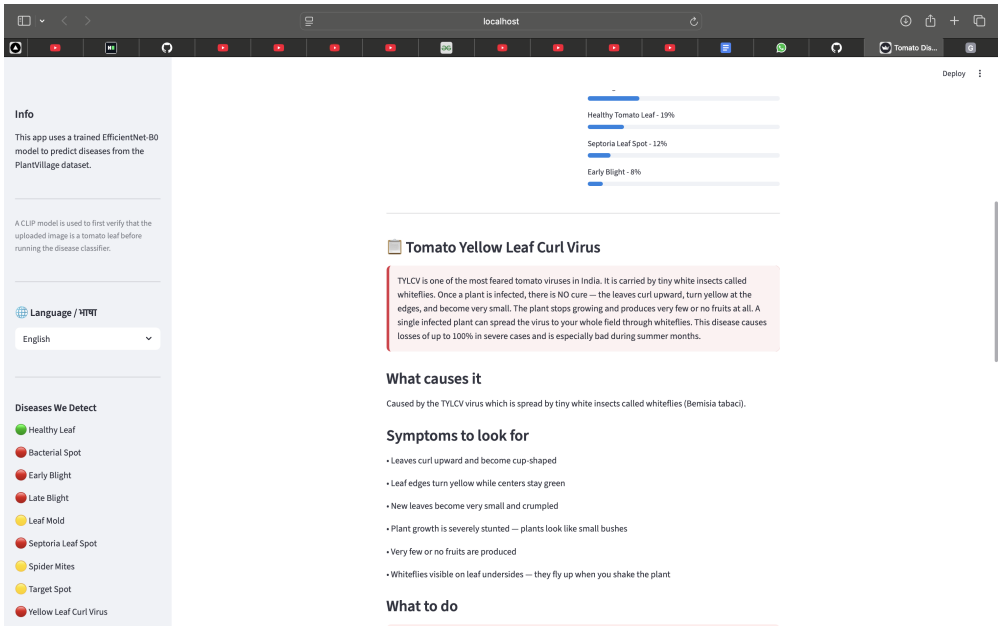


Figure 13: Tomato Yellow Leaf Curl Virus information card for the first tied diagnosis

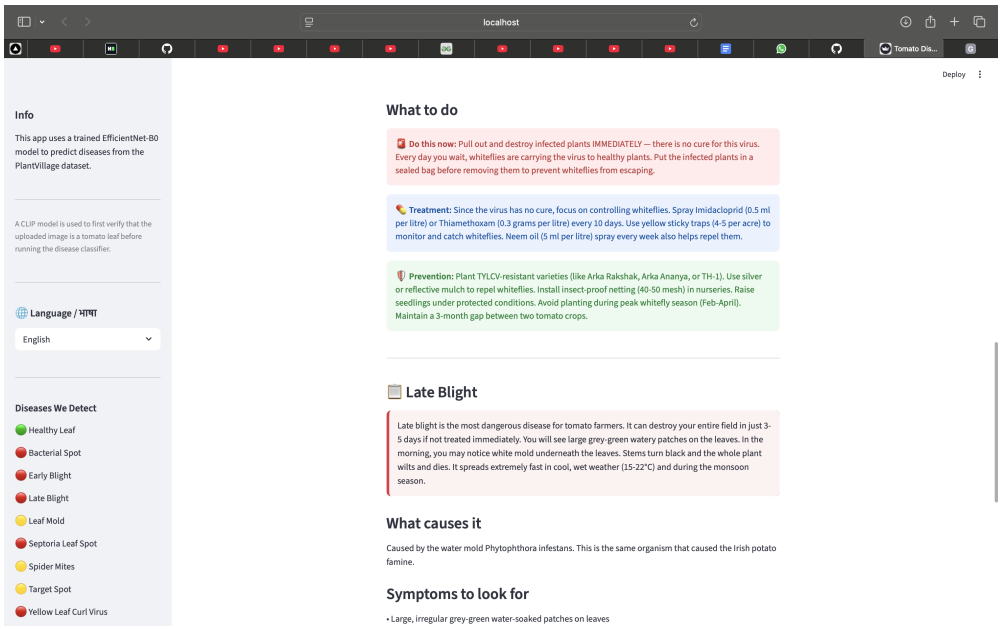


Figure 14: Treatment recommendations for TYLCV; Late Blight card follows below

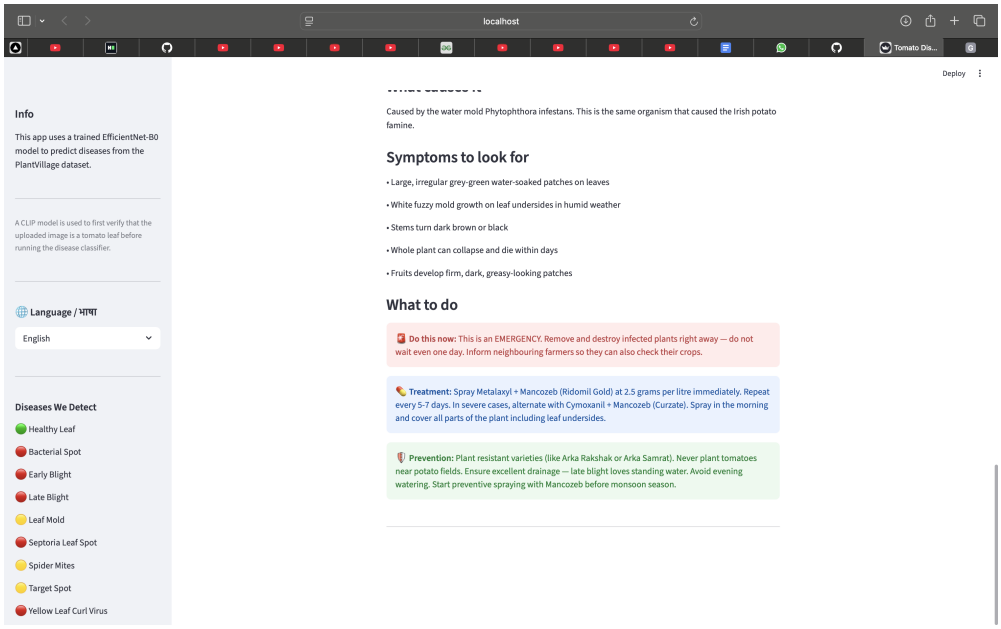
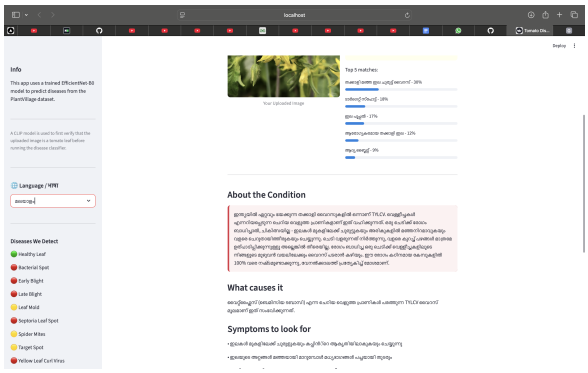


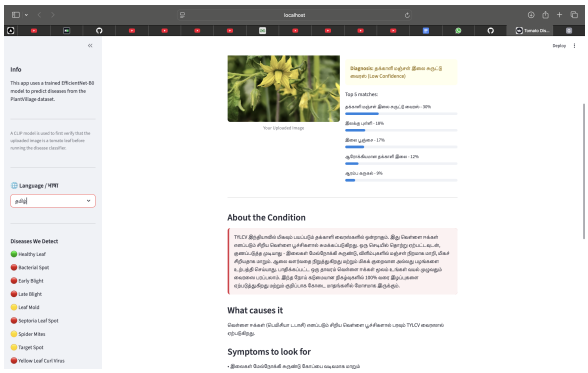
Figure 15: Late Blight information card, the second diagnosis in the tied result

6.7. Multi-Language Translation

When the user selects a regional language from the sidebar dropdown, every field the diagnosis label, top-5 class names, description, cause, symptoms, and treatment steps is immediately replaced with the pre-stored translation. No network request is made.

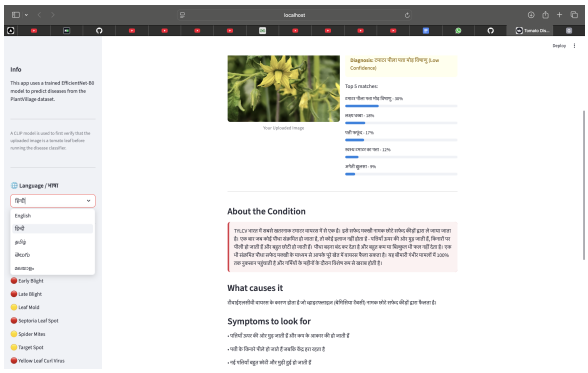


(a) Language dropdown showing all five options and Malayalam translation

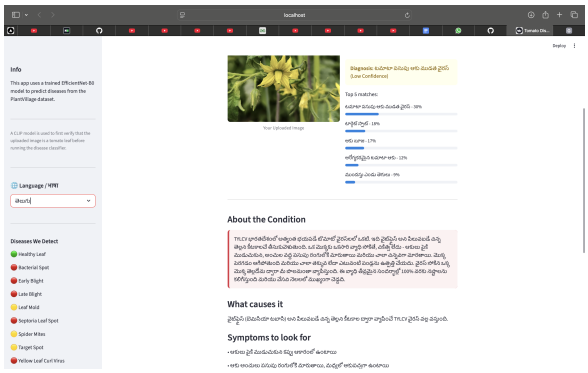


(b) Output fully translated to Tamil

Figure 16: Language selector and translated output



(a) Hindi translation



(b) Telugu translation

Figure 17: Disease output in Hindi and Telugu; all fields including top-5 class names are translated

7. Ablation Study

7.1. Effect of Removing the CLIP Gate

The most consequential design question is whether the CLIP validation stage is necessary at all. To answer this, we ran the EfficientNet-B0 classifier *without* the CLIP gate i.e., every image uploaded by the user is passed directly to the disease classifier regardless of its content.

Figure 18 illustrates the failure mode that results. A photograph of a notebook page containing handwritten mathematics is submitted to the classifier. Without CLIP to intercept it, the model confidently returns **Tomato Yellow Leaf Curl Virus (68% confidence)** as its top prediction, with Late Blight and Septoria Leaf Spot also appearing in the top-5. The classifier has no notion of "invalid input" it must always return one of its ten output classes, and it does so even when the input bears no visual relationship to a tomato leaf.

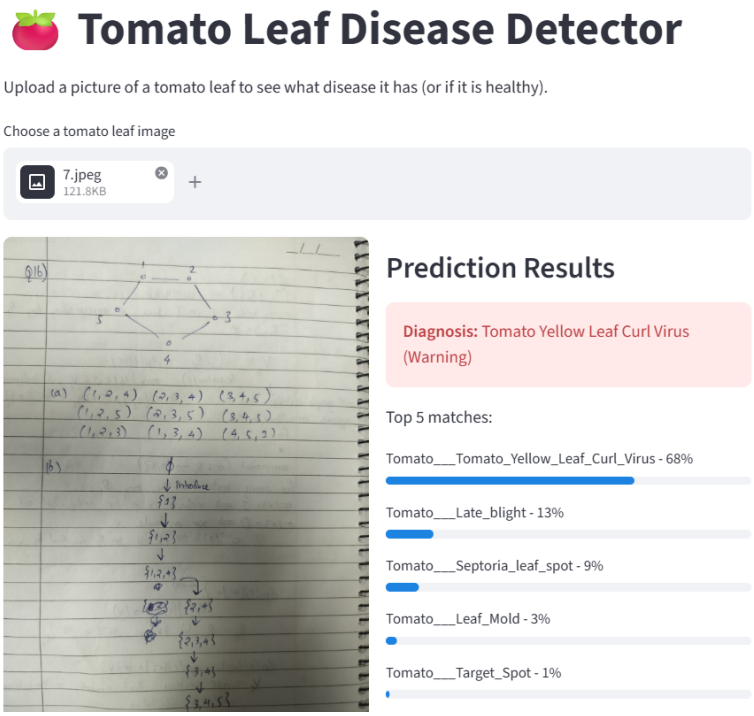


Figure 18: Without the CLIP gate: a notebook page with handwritten mathematics is classified as *Tomato Yellow Leaf Curl Virus* at 68% confidence. The classifier hallucinate a plausible-sounding disease label for any input.

This is a critical safety failure in an agricultural decision-support context. A farmer who accidentally photographs the wrong object their hand, the ground, a printed label would receive a confident disease diagnosis and potentially apply pesticides or destroy crops unnecessarily.

7.2. Quantitative Ablation Summary

Table 4 summarises the effect of each major design decision, including the CLIP gate and other choices made during development.

Table 4: Ablation: effect of individual design decisions on system behaviour

Configuration	**Metric Affected**	**Observed Effect**
No CLIP gate (classifier receives all inputs)	Invalid input handling	Classifier always produces a disease label; a notebook page, banana leaf, or photo of a car all return confident (but incorrect) tomato disease diagnoses (see Fig. 18)
Binary CLIP gate (hard 50% threshold)	User experience on borderline inputs	Blurry tomato photos and non-standard leaf angles frequently rejected; farmer has no recourse
Three-tier CLIP gate (20%–50% warn zone)	User agency	Borderline inputs surfaced with confidence score; user can continue or re-upload
Phase 1 only (head training, frozen backbone)	Validation accuracy	≈89% after one epoch; backbone features do not adapt to disease textures
Phase 2 only (full fine-tuning from epoch 1)	Training stability	Loss spikes in early epochs as large gradients corrupt pretrained representations
Two-phase fine-tuning (Phase 1 + Phase 2)	Validation accuracy	≈96% after three epochs; stable training curve
No label smoothing ($\epsilon = 0$)	Confidence calibration	Model assigns near-100% confidence to many predictions; ambiguity detection rarely triggers
Label smoothing ($\epsilon = 0.1$)	Confidence calibration	Predictions are better calibrated; low-confidence and tied-prediction states trigger correctly
Runtime translation (API)	Latency + connectivity	1–3 second delay per language switch; fails under poor network conditions
Pre-generated JSON translations	Latency + connectivity	Instant switching; fully offline

The ablation highlights three key findings. First, **the CLIP gate is essential for safety**: without it, the classifier confidently hallucinates disease labels for any image content. This is the most critical finding and directly motivated the two-stage pipeline design. Second, label smoothing directly affects how often the ambiguity-handling UI states activate , without it, the tied-prediction warning would almost never trigger because one class would always dominate at high confidence even when visual evidence is genuinely ambiguous. Third, the two-phase training strategy contributes a meaningful accuracy

delta (≈7 percentage points) over Phase 1 alone, consistent with the broader transfer learning literature and confirming that EfficientNet's ImageNet features require domain adaptation for fine-grained plant pathology.

8. Project Structure

Table 5: File structure and responsibilities

File	Purpose
`download_data.py`	Downloads PlantVillage from HuggingFace and extracts tomato classes
`train.py`	Two-phase EfficientNet-B0 fine-tuning; saves best checkpoint by validation accuracy
`app.py`	Streamlit app — CLIP gate, classification, disease UI, language switching
`disease_info.json`	Disease database: descriptions, symptoms, and treatments in five languages
`checkpoints/`	Saved model weights (`best_model.pth`)
`requirements.txt`	Python dependencies
`training-colab.ipynb`	Colab notebook for GPU-accelerated training

9. Technical Details

9.1. Dependencies

Table 6: Key dependencies and their roles

Library	Version	Role
`torch` + `torchvision`	≥2.0	Deep learning framework and image transforms
`timm`	≥0.9	EfficientNet-B0 with ImageNet pre-trained weights
`transformers`	≥4.30	CLIP model for zero-shot image verification
`streamlit`	≥1.30	Web application framework
`pillow`	≥9.0	Image loading and preprocessing

9.2. How to Run

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Download dataset (only required for training)
python download_data.py

# 3. Fine-tune the model (or use a pre-trained checkpoint)
```

```
python train.py --epochs 3 --batch_size 32

# 4. Launch the web app
streamlit run app.py --server.port 8502
```

10. Design Decisions

10.1. Why CLIP as a Gate?

Without a validation step, EfficientNet-B0 will produce a disease prediction for any image regardless of content a photograph of a car or a person would still return one of the ten tomato disease labels. CLIP's zero-shot capability makes it possible to screen inputs without training a separate binary classifier, and the approach generalises well to the wide variety of non-leaf images a farmer might accidentally upload.

10.2. Why Three Tiers Instead of a Binary Gate?

A strict pass/fail threshold would reject borderline inputs that a farmer might genuinely want to analyse blurry photos, partially visible leaves, or leaves from closely related solanaceous plants. The three-tier design preserves user agency: in the uncertain zone, the confidence score is surfaced and the user decides whether to proceed. For a rural user who cannot easily retake the photo (poor lighting, time pressure), this is meaningfully better than a hard rejection.

10.3. Why EfficientNet-B0?

EfficientNet-B0 has 5.3 million parameters and achieves accuracy competitive with much larger architectures on image classification benchmarks. The smaller model size reduces inference latency and server memory requirements, which matters if the application is ever deployed on a shared or resource-constrained host. The two-phase fine-tuning (freeze then unfreeze) is a standard technique to prevent catastrophic forgetting of the pretrained feature representations.

10.4. Why Pre-Generated Translations?

A runtime translation API would add latency, introduce a hard dependency on network connectivity, and potentially incur cost per request. Pre-generating all translations and bundling them in a JSON file means language switching is instant and the app remains fully functional in areas with no data connection exactly the conditions under which the target users are most likely to be operating.

11. Limitations

Several limitations are worth acknowledging.

Domain gap. The classifier is trained exclusively on PlantVillage, which contains controlled-environment images with plain backgrounds. While augmentation partially bridges this gap, accuracy on genuine field images is expected to be lower than the 96% reported on the PlantVillage validation set. A field validation study with farmer-captured images is needed to quantify this drop.

Class imbalance. The 14× imbalance between the largest and smallest classes (Yellow Leaf Curl Virus vs. Mosaic Virus) is not explicitly addressed during training beyond relying on cross-entropy loss. Weighted sampling or focal loss would likely improve recall on minority classes.

CLIP threshold sensitivity. The 20% and 50% thresholds for the gate were chosen through informal testing rather than a principled threshold search on a held-out set of non-leaf images. A systematic evaluation over a range of thresholds with precision–recall analysis would produce more reliable operating points.

Single-leaf assumption. The system assumes the uploaded image contains a single leaf in reasonable focus. Images with multiple overlapping leaves, stems, or fruit are not explicitly handled, and the classifier will attempt to produce a prediction regardless.

No severity progression. The system classifies disease type but not disease stage. A leaf in the early stage of Late Blight looks very different from one in an advanced stage, and the treatment urgency differs accordingly. A regression output for severity stage would add practical value but would require a more granular annotation scheme.

Mosaic Virus underrepresentation. With only 373 training samples, the classifier is at a disadvantage for this class. Supplementing with synthetic data via diffusion models or GAN-based augmentation could address this.

12. References

References

[1] Hughes, D. P. and Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv:1511.08060.

[2] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). *Learning transferable visual models from natural language supervision*. Proceedings of ICML 2021.

[3] Tan, M. and Le, Q. V. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*. Proceedings of ICML 2019.

[4] Wightman, R. (2019). *PyTorch Image Models (timm)*. `https://github.com/rwightman/pytorch-image-models`.

[5] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. (2016). *Rethinking the inception architecture for computer vision*. Proceedings of CVPR 2016.